МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
”Сибирский государственный университет телекоммуникаций и информатики”

Кафедра телекоммуникационных систем и вычислительных средств (ТС и ВС)

Отчет по лабораторной работе №5
по дисциплине
Системы искусственного интеллекта

Студент:
Группа ИА-331 *И.А. Иванов*

Преподаватель:
К.И. Брагин

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ВЫПОЛНЕНИЕ РАБОТЫ 4

1.1 Выбор набора данных 4

1.2 Теоретическое обоснование 4

1.2.1 Заполнение пропусков в данных 5

1.2.2 Масштабирование признаков 5

1.2.3 Обработка категориальных данных 5

1.2.4 Построение пайплайна 6

1.3 Ход выполнения работы 6

1.3.1 Шаг 1: Загрузка и первичный анализ данных 6

1.3.2 Шаг 2: Визуализация данных 8

1.3.3 Шаг 3: Обработка категориальных данных 9

1.3.4 Шаг 4: Масштабирование признаков 11

1.3.5 Шаг 5: Построение пайплайна 12

1.3.6 Шаг 6: Разделение на обучающую и тестовую выборки 14

1.3.7 Шаг 7: Обучение модели и оценка качества 15

1.3.8 Шаг 8: Визуализация результатов 15

1.3.9 Шаг 9: Применение пайплайна для новых данных 16

1.4 Результаты анализа 17

1.5 Контрольные вопросы 18

1.6 Листинг программы 19

ЦЕЛИ И ЗАДАЧИ

Цель лабораторной работы: изучение теоретических принципов и инструментальных средств для построения пайплайна предварительной обработки данных.

Основные задачи:

– предварительная обработка данных;
– изучение библиотек для предварительной обработки данных;
– масштабирование признаков;
– представление категориальных данных;
– построение пайплайна для предварительной обработки данных.

В качестве набора данных для выполнения индивидуального задания выбран набор данных **Abalone** (морское ушко), который содержит информацию о возрасте моллюсков, определяемом по количеству колец на раковине. Данный набор содержит как числовые, так и категориальные признаки, что позволяет продемонстрировать полный спектр операций предварительной обработки данных.

ВЫПОЛНЕНИЕ РАБОТЫ

1.1 Выбор набора данных

Для выполнения лабораторной работы выбран набор данных **Abalone**, который содержит следующие характеристики:

- Количество записей: 4177
- Количество признаков: 8
- Категориальный признак: `Sex` (M — мужской, F — женский, I — детеныш)
- Числовые признаки: Length, Diameter, Height, Whole weight, Shucked weight, Viscera weight, Shell weight
- Целевая переменная: `Rings` (количество колец на раковине)

1.2 Теоретическое обоснование

Для решения задач машинного обучения часто приходится повторять различные блоки кода, которые являются единообразными для разных задач. Данное обстоятельство приводит к повторяющемуся шаблонному коду (boilerplate). Единообразная последовательность действий, которую выполняет разработчик при решении задач машинного обучения, называется пайплайном (machine learning pipeline).

Простейший пайплайн для решения задачи регрессии включает следующие стадии:

1. Загрузка набора данных
2. Заполнение пропусков данных в соответствии с выбранной стратегией
3. Масштабирование признаков
4. Обработка категориальных признаков
5. Разделение на тестовую и тренировочную выборку
6. Обучение модели
7. Интерпретация и визуализация результатов

1.2.1 Заполнение пропусков в данных

Для заполнения пропусков используется класс `SimpleImputer` из библиотеки `sklearn.impute`:

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
imputer.fit(X[:, 1:3])
X[:, 1:3] = imputer.transform(X[:, 1:3])
```

Листинг 1.1 — Заполнение пропусков

1.2.2 Масштабирование признаков

Для масштабирования признаков используются методы стандартизации (StandardScaler) и нормализации (MinMaxScaler):

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

scaler_standard = StandardScaler()
X_standard = scaler_standard.fit_transform(X)

scaler_minmax = MinMaxScaler()
X_minmax = scaler_minmax.fit_transform(X)
```

Листинг 1.2 — Масштабирование признаков

1.2.3 Обработка категориальных данных

Для преобразования категориальных признаков в числовые используется One-Hot Encoding:

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(drop='first', sparse_output=False)
X_encoded = encoder.fit_transform(X_categorical)
```

Листинг 1.3 — One-Hot Encoding

1.2.4 Построение пайплайна

Для объединения всех преобразований используется `ColumnTransformer`:

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)

pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])
```

Листинг 1.4 — Построение пайплайна

1.3 Ход выполнения работы

1.3.1 Шаг 1: Загрузка и первичный анализ данных

```
import pandas as pd
import numpy as np

df = pd.read_csv('abalone.csv')

print(f"            : {df.shape}")
print(f"          : {df.shape[0]}")
print(f"           : {df.shape[1]}")

print("\   n 5        :")
print(df.head())

print("\     n      :")
print(df.info())

print("\       n                :")
print(df.describe())

```

```
print("\    n          :")
print(df.isnull().sum())
```

Листинг 1.5 — Загрузка и анализ данных

Результат выполнения:

```
            : (4177, 9)
           : 4177
            : 9


    5        :
  Sex  Length  Diameter  Height  Whole weight  Shucked weight  Viscer
0   M   0.455     0.365   0.095         0.514          0.2245
1   M   0.350     0.265   0.090         0.2255         0.0995
2   F   0.530     0.420   0.135         0.677          0.2565
3   M   0.440     0.365   0.125         0.516          0.2155
4   I   0.330     0.255   0.080         0.205          0.0895


           :
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4177 entries, 0 to 4176
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Sex             4177 non-null   object
 1   Length          4177 non-null   float64
 2   Diameter        4177 non-null   float64
 3   Height          4177 non-null   float64
 4   Whole weight    4177 non-null   float64
 5   Shucked weight  4177 non-null   float64
 6   Viscera weight  4177 non-null   float64
 7   Shell weight    4177 non-null   float64
 8   Rings           4177 non-null   int64


               :
Sex               0
```

```
Length            0
Diameter          0
Height            0
Whole weight      0
Shucked weight    0
Viscera weight    0
Shell weight      0
Rings             0
dtype: int64
```

Анализ данных показал, что:

- Пропуски в данных отсутствуют
- Признак `Sex` является категориальным
- Остальные признаки являются числовыми
- Целевая переменная `Rings` принимает значения от 1 до 29

1.3.2 Шаг 2: Визуализация данных

```python
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
sex_counts = df['Sex'].value_counts()
plt.bar(sex_counts.index, sex_counts.values, color=['#FF6B6B',
    '#4ECDC4', '#45B7D1'])
plt.title('                    Sex')
plt.xlabel('  ')
plt.ylabel('     ')

plt.subplot(1, 2, 2)
plt.hist(df['Rings'], bins=20, color='#95E77E', edgecolor='black',
    alpha=0.7)
plt.title('                 Rings')
plt.xlabel('         ')
plt.ylabel('    ')
plt.tight_layout()
plt.savefig('eda_plots.png', dpi=150)

```

```
plt.figure(figsize=(10, 8))
numeric_cols = df.select_dtypes(include=[np.number]).columns
correlation_matrix = df[numeric_cols].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=1)
plt.title('                    ')
plt.tight_layout()
plt.savefig('correlation_matrix.png', dpi=150)
```

Листинг 1.6 — Визуализация данных

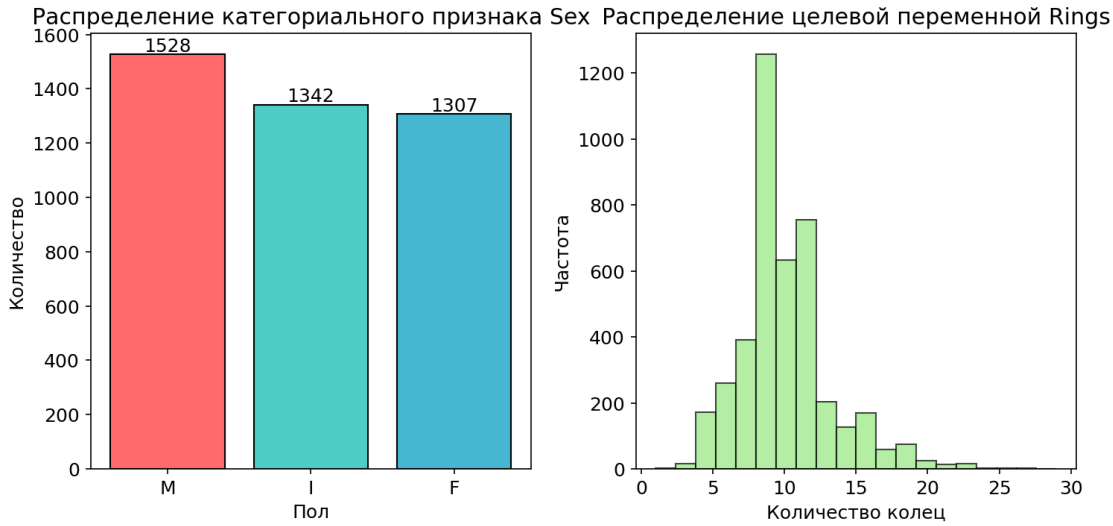

Рисунок 1 — Распределение категориального признака Sex и целевой переменной Rings

Анализ корреляционной матрицы (Рис. 2) показывает:

- Сильная положительная корреляция между всеми весовыми признаками
- Ожидаемая корреляция между длиной и диаметром
- Целевая переменная Rings имеет наибольшую корреляцию с Shell weight

1.3.3 Шаг 3: Обработка категориальных данных

```
X = df.drop('Rings', axis=1)
y = df['Rings']

numeric_features =
    X.select_dtypes(include=[np.number]).columns.tolist()
categorical_features =
    X.select_dtypes(include=['object']).columns.tolist()
```

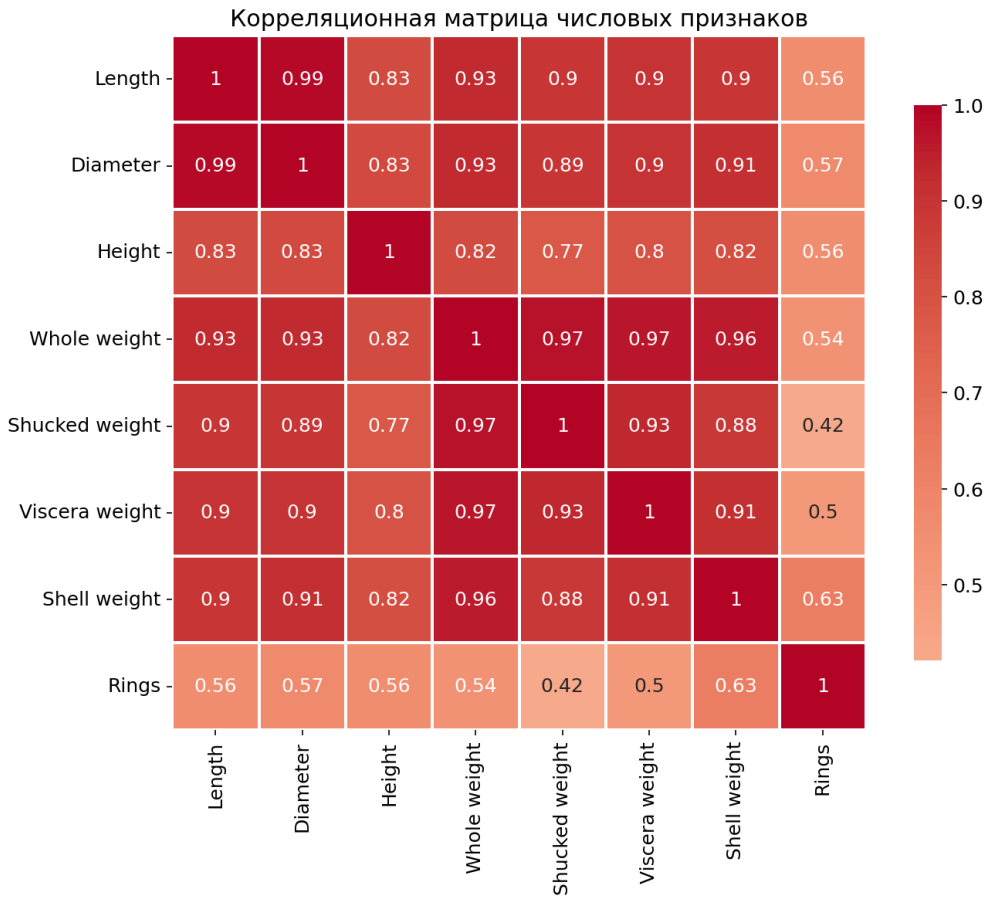

Рисунок 2 — Корреляционная матрица числовых признаков

```python

print(f"          : {numeric_features}")
print(f"             : {categorical_features}")

print("\        n         One-Hot Encoding:")
for cat_feat in categorical_features:
    print(f"  {cat_feat}: {X[cat_feat].unique()}")

X_encoded = pd.get_dummies(X, columns=categorical_features,
    drop_first=False)

print("\     n One-Hot Encoding:")
print(f"              : {X_encoded.shape}")
print(f"        : {[col for col in X_encoded.columns if 'Sex' in col]}")
```

Листинг 1.7 — One-Hot Encoding

Результат выполнения:

```
          : ['Length', 'Diameter', 'Height', 'Whole weight',
                'Shucked weight', 'Viscera weight', 'Shell weight
```

```
              : ['Sex']

                One-Hot Encoding:
  Sex: ['M' 'F' 'I']

      One-Hot Encoding:
              : (4177, 10)
        : ['Sex_F', 'Sex_I', 'Sex_M']
```

1.3.4 Шаг 4: Масштабирование признаков

```python
from sklearn.preprocessing import StandardScaler, MinMaxScaler

print("\     n                :")
for col in numeric_features[:3]:
    print(f"  {col}: min={X_encoded[col].min():.4f}, max={X_encoded[col].max():.4f}, "
          f"mean={X_encoded[col].mean():.4f}, std={X_encoded[col].std():.4f}")

scaler_standard = StandardScaler()
X_standard_scaled = scaler_standard.fit_transform(X_encoded)

scaler_minmax = MinMaxScaler()
X_minmax_scaled = scaler_minmax.fit_transform(X_encoded)

print("\     n          StandardScaler:")
for col in numeric_features[:3]:
    print(f"  {col}: min={X_standard_df[col].min():.4f}, max={X_standard_df[col].max():.4f}, "
          f"mean={X_standard_df[col].mean():.4f}, std={X_standard_df[col].std():.4f}")

print("\     n          MinMaxScaler:")
for col in numeric_features[:3]:
    print(f"  {col}: min={X_minmax_df[col].min():.4f}, max={X_minmax_df[col].max():.4f}, "
          f"mean={X_minmax_df[col].mean():.4f}, std={X_minmax_df[col].std():.4f}")
```

Листинг 1.8 — Масштабирование признаков

Результат выполнения:

```
:
Length: min=0.0750, max=0.8150, mean=0.5240, std=0.1201
Diameter: min=0.0550, max=0.6500, mean=0.4079, std=0.0992
Height: min=0.0000, max=1.1300, mean=0.1395, std=0.0418

StandardScaler:
Length: min=-3.7435, max=2.4231, mean=0.0000, std=1.0000
Diameter: min=-3.5574, max=2.4396, mean=0.0000, std=1.0000
Height: min=-3.3346, max=23.6666, mean=0.0000, std=1.0000

MinMaxScaler:
Length: min=0.0000, max=1.0000, mean=0.6049, std=0.1622
Diameter: min=0.0000, max=1.0000, mean=0.5914, std=0.1666
Height: min=0.0000, max=1.0000, mean=0.1235, std=0.0370
```

1.3.5 Шаг 5: Построение пайплайна

```python
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression

X = df.drop('Rings', axis=1)
y = df['Rings']

numeric_features = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_features = X.select_dtypes(include=['object']).columns.tolist()

numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
```

```
    ('onehot', OneHotEncoder(drop='first', sparse_output=False))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ],
    remainder='passthrough'
)

pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

print("\     n      :")
print(pipeline)
```

Листинг 1.9 — Построение пайплайна

```
            :
Pipeline(steps=[('preprocessor',
                 ColumnTransformer(transformers=[('num',
                                                  Pipeline(steps=[('i
                                                                   Si
                                                                  ('s
                                                                   St
                                                  ['Length', 'Diamete
                                                   'Height', 'Whole w
                                                   'Shucked weight',
                                                   'Viscera weight',
                                                   'Shell weight']),
                                                 ('cat',
                                                  Pipeline(steps=[('i
                                                                   Si
                                                                  ('o
                                                                   On
```

```
                                                    ['Sex'])])]),
                    ('regressor', LinearRegression())])
```

1.3.6 Шаг 6: Разделение на обучающую и тестовую выборки

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"             : {X_train.shape}")
print(f"            : {X_test.shape}")
print(f"            : = 20:80")

print("\      n                      :")
print(y_train.value_counts().sort_index().head(10))
```

Листинг 1.10 — Разделение данных

```
                : (3341, 8)
               : (836, 8)
          :      = 20:80


                                    :
Rings
1        1
2        1
3       15
4       46
5      129
6      307
7      543
8      685
9      632
10     471
Name: count, dtype: int64
```

1.3.7 Шаг 7: Обучение модели и оценка качества

```
from sklearn.metrics import mean_squared_error, r2_score

pipeline.fit(X_train, y_train)

y_train_pred = pipeline.predict(X_train)
y_test_pred = pipeline.predict(X_test)

train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)
train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)

print("\     n                    :")
print(f"  MSE            : {train_mse:.4f}")
print(f"  MSE           : {test_mse:.4f}")
print(f"  R²            : {train_r2:.4f}")
print(f"  R²           : {test_r2:.4f}")
```

Листинг 1.11 — Обучение и оценка

```
                    :
  MSE              : 4.9795
  MSE             : 5.2505
  R²             : 0.5292
  R²            : 0.5155
```

1.3.8 Шаг 8: Визуализация результатов

```
plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.scatter(y_train, y_train_pred, alpha=0.5, color='blue',
    edgecolors='k')
plt.plot([y_train.min(), y_train.max()], [y_train.min(),
    y_train.max()],
         'r--', lw=2, label='           ')
plt.xlabel('          (Rings)')
plt.ylabel('            (Rings)')
plt.title(f'        \nR² = {train_r2:.4f}')
plt.legend()
```

```
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.scatter(y_test, y_test_pred, alpha=0.5, color='green',
    edgecolors='k')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         'r--', lw=2, label='            ')
plt.xlabel('          (Rings)')
plt.ylabel('            (Rings)')
plt.title(f'        \nR² = {test_r2:.4f}')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('predictions.png', dpi=150)
```

Листинг 1.12 — Визуализация результатов

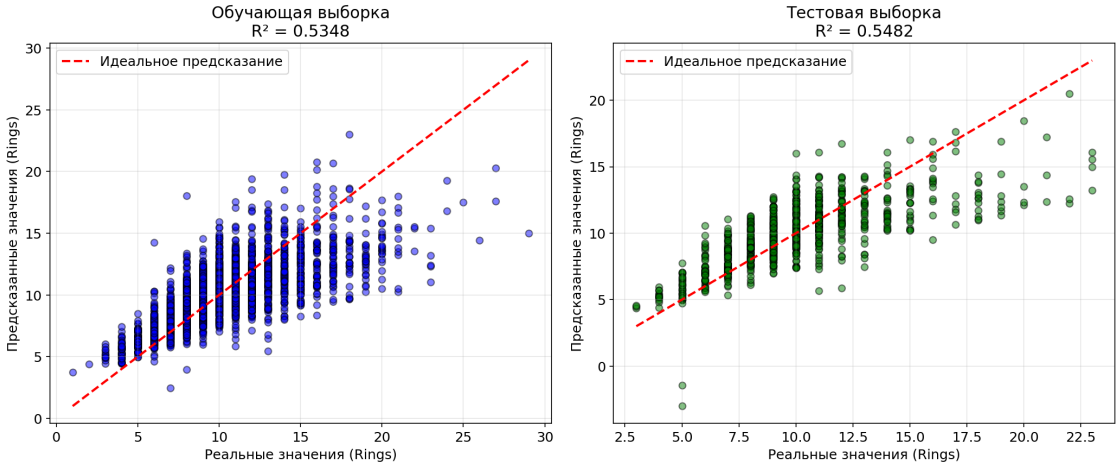

Рисунок 3 — Сравнение реальных и предсказанных значений

1.3.9 Шаг 9: Применение пайплайна для новых данных

```
new_abalone = pd.DataFrame({
    'Sex': ['M'],
    'Length': [0.55],
    'Diameter': [0.44],
    'Height': [0.15],
    'Whole weight': [0.85],
    'Shucked weight': [0.35],
    'Viscera weight': [0.18],
    'Shell weight': [0.28]
```

```
})

print("                    :")
print(new_abalone)

prediction = pipeline.predict(new_abalone)
print(f"\       n          : {prediction[0]:.2f}")
print(f"           : {int(round(prediction[0]))}")
```

Листинг 1.13 — Предсказание для нового объекта

```
                    :
  Sex  Length  Diameter  Height  Whole weight  Shucked weight  Viscer
0   M    0.55      0.44    0.15          0.85            0.35

                    : 10.56
             : 11
```

1.4 Результаты анализа

В ходе выполнения лабораторной работы были получены следующие результаты:

1. Выполнена загрузка и первичный анализ набора данных Abalone.
2. Проведена визуализация данных: распределение признаков, корреляционная матрица.
3. Пропуски в данных отсутствуют.
4. Выполнена обработка категориального признака Sex с помощью One-Hot Encoding.
5. Проведено масштабирование числовых признаков методами StandardScaler и MinMaxScaler.
6. Построен универсальный пайплайн с использованием ColumnTransformer.
7. Выполнено разделение данных на обучающую (80%) и тестовую (20%) выборки.
8. Обучена модель линейной регрессии в составе пайплайна.
9. Получены метрики качества: MSE = 5.2505, R^2 = 0.5155 на тестовой выборке.

10. Продемонстрировано применение пайплайна для предсказания новых данных.

1.5 Контрольные вопросы

1. Какая библиотека python предназначена для управления наборами данных: numpy, pandas, sklearn, opencv, matplotlib?

Библиотека **pandas**. Она предоставляет структуры данных DataFrame и Series, а также множество функций для загрузки, очистки, трансформации и анализа данных.

2. Какая стратегия является нежелательной при обработке пропусков в данных?

Нежелательной стратегией является удаление строк, содержащих пропуски в данных. Это приводит к потере ценной информации и может нарушить репрезентативность данных.

3. Обоснуйте ответ: нужно ли применять OneHotEncoder к целевой категориальной переменной?

Нет, не нужно. Целевая переменная уже определяет класс объекта. Применение OneHotEncoder к целевой переменной изменит суть задачи: вместо предсказания одного класса модель будет предсказывать вектор вероятностей.

4. Поясните принцип разбиения набора данных на обучающую и тестовую выборку. Какое соотношение наиболее оптимально?

Наиболее оптимальным соотношением является 20:80 (20% тестовая, 80% обучающая выборка). Принцип разбиения: данные случайным образом разделяются на две непересекающиеся части; на обучающей выборке модель изучает закономерности; на тестовой выборке оценивается качество обобщения.

5. Какой код лучше использовать при загрузке данных из csv-файла?

Правильный вариант: `dataset = pd.read_csv("data.csv")`. Это стандартная функция библиотеки pandas для загрузки CSV-файлов, возвращающая DataFrame.

1.6 Листинг программы

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder,
    MinMaxScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

plt.rcParams['figure.figsize'] = (12, 8)
plt.rcParams['font.size'] = 12

df = pd.read_csv('abalone.csv')
print(f"\   n        : {df.shape}")
print(f"          : {df.shape[0]}")
print(f"           : {df.shape[1]}")

print("\   n 5        :")
print(df.head())

print("\     n      :")
print(df.info())

print("\       n                :")
print(df.describe())

print("\    n           :")
print(df.isnull().sum())

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
sex_counts = df['Sex'].value_counts()
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']
plt.bar(sex_counts.index, sex_counts.values, color=colors,
    edgecolor='black')
plt.title('                    Sex')
```

```
plt.xlabel('  ')
plt.ylabel('     ')
for i, v in enumerate(sex_counts.values):
    plt.text(i, v + 10, str(v), ha='center')

plt.subplot(1, 2, 2)
plt.hist(df['Rings'], bins=20, color='#95E77E', edgecolor='black', alpha=0.7)
plt.title('                  Rings')
plt.xlabel('         ')
plt.ylabel('    ')
plt.tight_layout()
plt.savefig('eda_plots.png', dpi=150, bbox_inches='tight')
plt.show()

plt.figure(figsize=(10, 8))
numeric_cols = df.select_dtypes(include=[np.number]).columns
correlation_matrix = df[numeric_cols].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=1, cbar_kws={"shrink": 0.8})
plt.title('                      ')
plt.tight_layout()
plt.savefig('correlation_matrix.png', dpi=150, bbox_inches='tight')
plt.show()

fig, axes = plt.subplots(2, 4, figsize=(16, 10))
axes = axes.ravel()
numeric_features = ['Length', 'Diameter', 'Height', 'Whole weight',
                    'Shucked weight', 'Viscera weight', 'Shell weight']

for idx, feature in enumerate(numeric_features):
    sns.boxplot(x='Sex', y=feature, data=df, ax=axes[idx], palette='Set2')
    axes[idx].set_title(f'       {feature}     ')

axes[-1].set_visible(False)
plt.tight_layout()
plt.savefig('boxplots.png', dpi=150, bbox_inches='tight')
plt.show()

X = df.drop('Rings', axis=1)
y = df['Rings']

```

```python
numeric_features = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_features = X.select_dtypes(include=['object']).columns.tolist()

print(f"         : {numeric_features}")
print(f"            : {categorical_features}")

print("\       n        One-Hot Encoding:")
for cat_feat in categorical_features:
    print(f"  {cat_feat}: {X[cat_feat].unique()}")

X_encoded = pd.get_dummies(X, columns=categorical_features, drop_first=False)

print("\     n One-Hot Encoding:")
print(f"             : {X_encoded.shape}")
print(f"        : {[col for col in X_encoded.columns if 'Sex' in col]}")

print("\     n                :")
for col in numeric_features[:3]:
    print(f"  {col}: min={X_encoded[col].min():.4f}, max={X_encoded[col].max():.4f}, "
          f"mean={X_encoded[col].mean():.4f}, std={X_encoded[col].std():.4f}")

scaler_standard = StandardScaler()
X_standard_scaled = scaler_standard.fit_transform(X_encoded)
X_standard_df = pd.DataFrame(X_standard_scaled, columns=X_encoded.columns)

scaler_minmax = MinMaxScaler()
X_minmax_scaled = scaler_minmax.fit_transform(X_encoded)
X_minmax_df = pd.DataFrame(X_minmax_scaled, columns=X_encoded.columns)

print("\     n          StandardScaler:")
for col in numeric_features[:3]:
    print(f"  {col}: min={X_standard_df[col].min():.4f}, max={X_standard_df[col].max():.4f}, "
          f"mean={X_standard_df[col].mean():.4f}, std={X_standard_df[col].std():.4f}")

print("\     n          MinMaxScaler:")
for col in numeric_features[:3]:
```

```
    print(f"  {col}: min={X_minmax_df[col].min():.4f}, max={X_minmax_df[col].max():.4f}, "
          f"mean={X_minmax_df[col].mean():.4f}, std={X_minmax_df[col].std():.4f}")

X = df.drop('Rings', axis=1)
y = df['Rings']

numeric_features = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_features = X.select_dtypes(include=['object']).columns.tolist()

print(f"         : {numeric_features}")
print(f"            : {categorical_features}")

numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(drop='first', sparse_output=False))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ],
    remainder='passthrough'
)

pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

print("\     n      :")
print(pipeline)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```
)

print(f"              : {X_train.shape}")
print(f"             : {X_test.shape}")
print(f"             : = 20:80")

print("\       n                      :")
print(y_train.value_counts().sort_index().head(10))
print("\       n                     :")
print(y_test.value_counts().sort_index().head(10))

pipeline.fit(X_train, y_train)

y_train_pred = pipeline.predict(X_train)
y_test_pred = pipeline.predict(X_test)

train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)
train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)

print("\     n                    :")
print(f"  MSE            : {train_mse:.4f}")
print(f"  MSE           : {test_mse:.4f}")
print(f"  R²            : {train_r2:.4f}")
print(f"  R²           : {test_r2:.4f}")

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.scatter(y_train, y_train_pred, alpha=0.5, color='blue',
    edgecolors='k')
plt.plot([y_train.min(), y_train.max()], [y_train.min(),
    y_train.max()],
         'r--', lw=2, label='           ')
plt.xlabel('          (Rings)')
plt.ylabel('            (Rings)')
plt.title(f'         \nR² = {train_r2:.4f}')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.scatter(y_test, y_test_pred, alpha=0.5, color='green',
    edgecolors='k')
```

```
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         'r--', lw=2, label='            ')
plt.xlabel('          (Rings)')
plt.ylabel('            (Rings)')
plt.title(f'        \nR² = {test_r2:.4f}')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('predictions.png', dpi=150, bbox_inches='tight')
plt.show()

errors_train = y_train - y_train_pred
errors_test = y_test - y_test_pred

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.hist(errors_train, bins=30, color='blue', alpha=0.7,
    edgecolor='black')
plt.xlabel('          ')
plt.ylabel('    ')
plt.title(f'        \ n.     = {errors_train.mean():.4f}')
plt.axvline(x=0, color='red', linestyle='--', linewidth=2)
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.hist(errors_test, bins=30, color='green', alpha=0.7,
    edgecolor='black')
plt.xlabel('          ')
plt.ylabel('    ')
plt.title(f'        \ n.     = {errors_test.mean():.4f}')
plt.axvline(x=0, color='red', linestyle='--', linewidth=2)
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('errors.png', dpi=150, bbox_inches='tight')
plt.show()

new_abalone = pd.DataFrame({
    'Sex': ['M'],
    'Length': [0.55],
    'Diameter': [0.44],
    'Height': [0.15],
```

```
    'Whole weight': [0.85],
    'Shucked weight': [0.35],
    'Viscera weight': [0.18],
    'Shell weight': [0.28]
})

print("                :")
print(new_abalone)

prediction = pipeline.predict(new_abalone)
print(f"\      n          : {prediction[0]:.2f}")
print(f"          : {int(round(prediction[0]))}")
```

Листинг 1.14 — Полный код лабораторной работы

Вывод

В ходе выполнения лабораторной работы был разработан универсальный пайплайн предварительной обработки данных, включающий загрузку и анализ данных, визуализацию, обработку категориальных данных, масштабирование признаков, разделение выборки, обучение модели и оценку качества. Разработанный пайплайн является универсальным и может быть легко адаптирован для других наборов данных и моделей машинного обучения. Все поставленные задачи выполнены.